Git技术与使用流程

> 🧩 引言： Git是一个分布式版本控制系统， 用于快速高效处理各类项目
>
> - **版本控制**： 一种记录一个或若干个文件内容变化，以便将来查阅特定版本修订情况的系统
> - 软件开发中版本控制系统的**重要性**：回退到过去版本；查看版本变化细节；查看修改历史细节(提交者/改动地方)；查看功能缺陷报告

1. 基本介绍

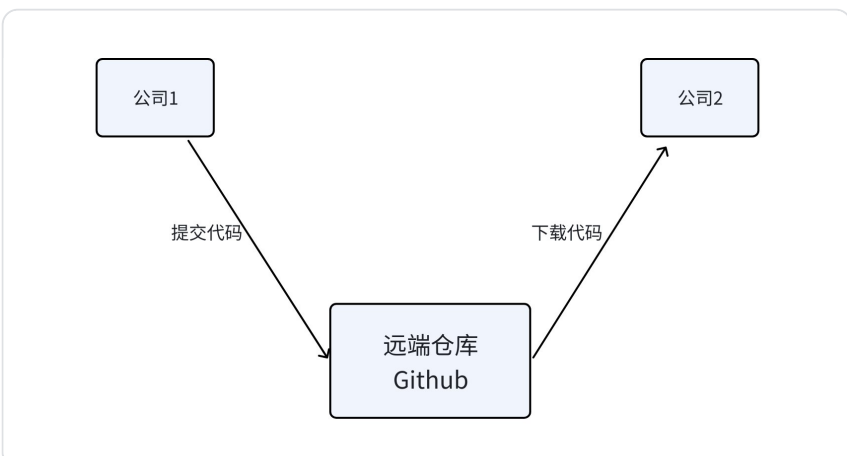
版本控制系统分类

1. 集中化版本控制系统 (SVN,CVS....)

- 特点：
  - 单一的集中管理的服务器，所有人可以向服务器上传/下载
  - 每个人都能在一定程度上看到项目中其他人的进度
  - 管理员掌控开发者权限，集中化管理
- 缺陷：
  - 中央服务器**单点故障：**所有人都无法提交更新，继续更改则不能保存
  - 必须联网访问中央服务器

2. 分布式版本控制系统 (Git)

- 特点：
  - 客户端不只提取最新版本的文件快照，而是把代码仓库**完整的镜像**下来 (**本地**有所有的**历史版本**，而非最新版本{SVD}) #游戏存档
  - 通过算法 使Git 占用的存储 仅比SVN多一点，但是存储的内容是SVN的几倍
  - 不用联网就可以访问过去版本
  - 可以指定和若干不同远端代码仓库进行交互：可以在同一个项目中与不同工作小组的人相互协作



  - 管理项目时，存放的不是版本与版本间的差异，存的是索引（所需的磁盘空间很少所以每个客户端都能放下整个项目的历史记录）

2. 使用流程

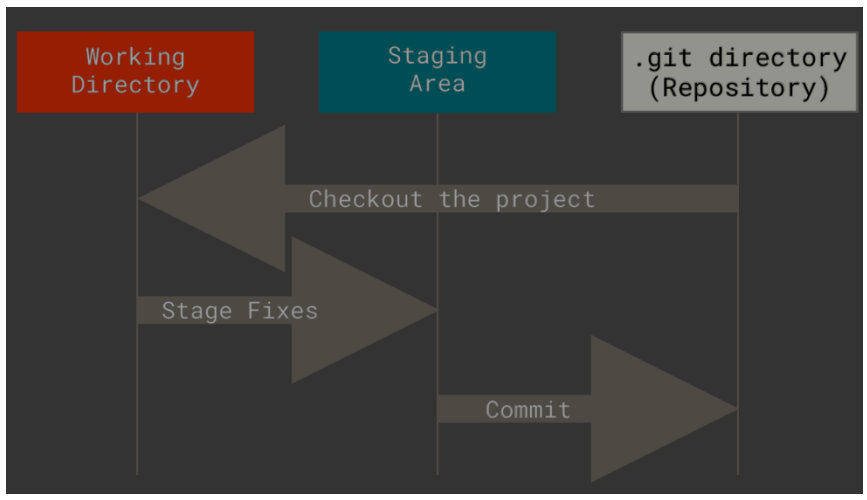
Website usage process

1. git-scm.com
2. "Learn"

Three stages

- There are three main stages at files: **modified, staged and commited**
  - Modified: Files have **been changed** but **not commit** to the **database**
  - Staged: Already have a "modified file" in current version to go to the next "commmit snapshot"
  - Commited: Data is safely stored in **local database**

Three main work sections:



1. Staging area ("index"): Stores information which **will** go into "next commit" (存储暂未commit的数据)
2. Git directory (Most important): Stores metadata and object database; 从另一台电脑克隆存储库时克隆的就是该内容

⭐Basic Git Work Flow

1. **Modify:** Modify files in the working tree 修改文件内容
2. **Stage: Selectively stage** just those changes you want to be part of your next commit, which **adds *only* those changes** to the staging area 选择下一步要commit的文件，并只把他们加到"Staging Area"中
3. **Commit:** Do a **commit**, which takes the files as **they are in the staging area** and stores that snapshot permanently to your **Git directory.** 提交在staging area中的文件(上一步必须先stage)，并永久存储在Git dir中

3. Command Line

3.1 Customize your environment

- Set your "user name" and "email address" ---每次有 upgrade 时都会调用该信息
  - Recommendation: Create a Github account and use the corresponding email
  - "User name":

```
代码块
```

```
git config --global user.name "John Doe"
```

- "Email address":

代码块

```
git config --global user.email johndoe@example.com
```

- You only need to do this **once** due to **"--global" 只用设置一次** **#override with different name/email**: run command without global

3.2 Editor Configuration

- Configure your default text Editor, or Git will use its systems' text editor 自行配置/使用系统默认

Example: use a different text editor, such as Emacs

```
git config --global core.editor emacs
```

- In the Windows system, the full path of exe. File should be supplied Windows系统中需要提供exe完整路径

代码块

```
git config --global core.editor "'C:/Program Files/Notepad++/notepad++.exe' -multiInst -notabbar -nosession -noPlugin"
"""-multiInst
 以多实例模式打开，也就是新开一个独立窗口，而不是复用已有窗口。
-notabbar
 不显示标签栏。
-nosession
 不加载上一次的会话内容，避免打开以前没关的文件。
-noPlugin
 启动时不加载插件，减少干扰"""
```

3.3 Change Default branch name

- When you **create a new repositor**y with the command: 创建新库

代码块

```
git init
#feedback: Initialised empty Git in repository
```

- Git will create a branch called "master" and you can **change its name** through the following code:修改默认branch名称

代码块

```
git config --global init.defaultBranch main
#Set main as the new default branch name
```

3.4 Check Setting

- You can check your configuration list by:

```
代码块
git config --list
#you can get the feedback like:
#user.name=John Doe
#user.email=johndoe@example.com
#color.status=auto
#color.branch=auto
#color.interactive=auto
#color.diff=auto
```

- You can also check for a specific value by:

```
代码块
git config <key>
#git config user.name
#John Doe
不再会出现等式
```

3.5 Getting help

- there are three equivalent ways to get the comprehensive manual page (manpage) **help** for any of the Git commands:

```
代码块
git help <verb>
git <verb> --help
man git-<verb>
#git help config
```

4. Git Base

4.1 Getting a Git Repository

- There are two methods to get a Git repository:

1. Take a local directory that is currently not under version control, and **turn it into** a Git repository
   a. What is "not under version control": A normal file without executing "git. init" or clone from other repository 自己建立的，Git 不知道其存在的文件夹 (不含有".Git")
   b. Initialise a repository:
      i. Go to this file's directory

```
代码块
#linux:
 cd /home/user/my_project
 #Windows:
 cd C:/Users/user/my_project
 #MacOS
 cd /Users/user/my_project
```

ii. Then you can type in: (creates a new **subdirectory** named `.git` that contains all of your necessary repository file) 这个文件通常是在隐藏文件夹中

代码块

```
git init
```

iii. **Get version control** of existing files:

代码块

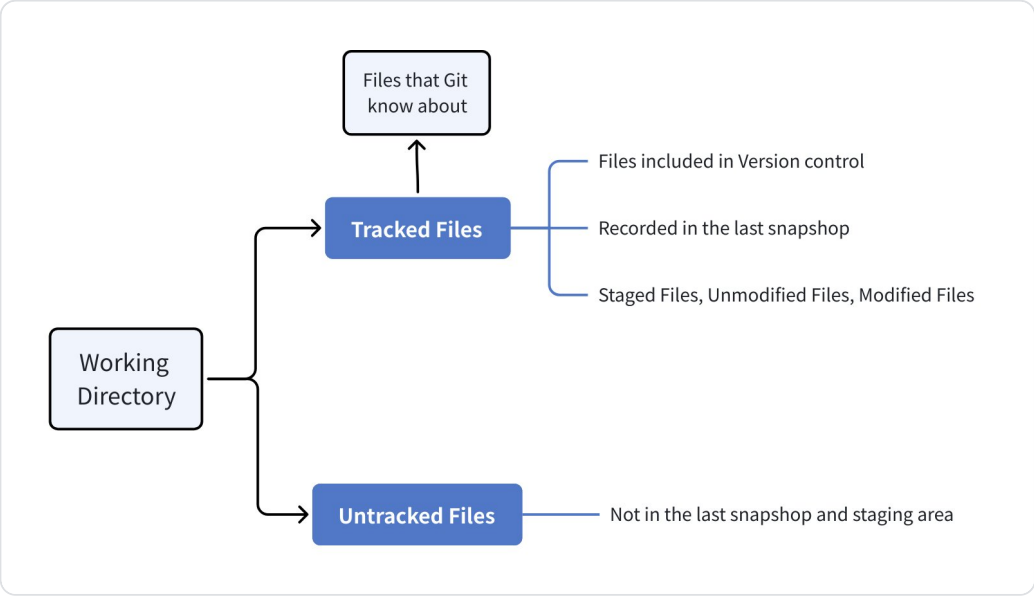
```
#track the corresponding file and put them into the staging area,跟踪文件并放入暂存区(commit候选列表)
#工作区——>暂存区
git add *.c  #所有的.c文件
git add LICENSE
#将git add进暂存区的文件正式提交，作为version 记录
#暂存区——>仓库历史
git commit -m 'Initial project version'
```

2. ***Clone*** an existing Git repository from elsewhere

代码块

```
git clone <url>
#Example: clone a repository on github called libgit2
# git clone https://github.com/libgit2/libgit2  用仓库原名在本地建立目录
# git clone https://github.com/libgit2/libgit2 mylibgit 在本地建立目录并且命名为mylibgit
```

4.2 Recording Changes to the Repository 记录变化到仓库



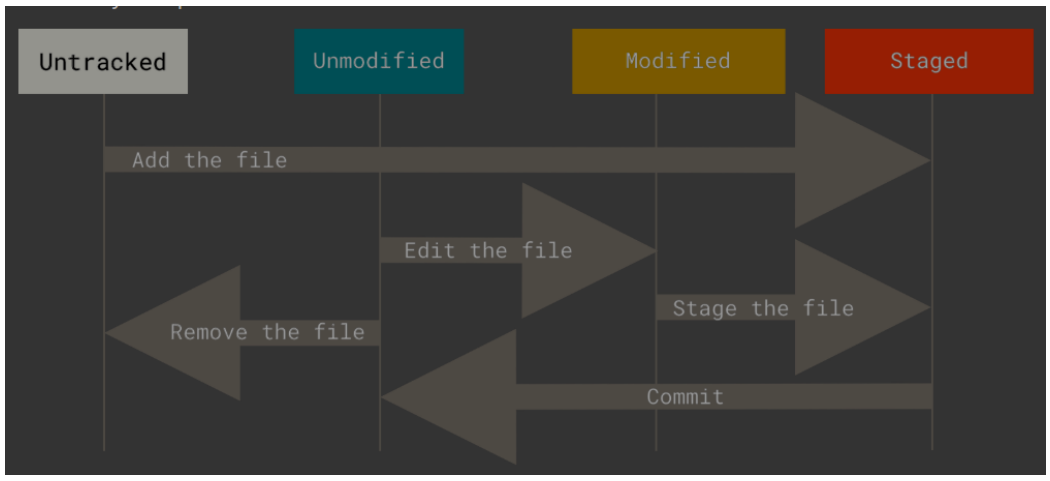
- When you first clone a repository, all of your files will be **tracked** and **unmodifie**d because Git just checked them out and you haven’t edited anything.
- As you edit files, Git sees them as modified, because you’ve **changed them since your last commit**. As you work, you **selectively stage** these modified files and then **commit** all those **staged changes**, and the cycle repeats.

/libgit2/libgit2 用仓库原名在本地建立目录

段竣瀚 4月14日 10:56

第一个 libgit2--组织/用户名，这个仓库在 Github 上属于谁

第二个 libgit2--仓库名，在这个用户下的哪一个仓库



- 查看 Current Status of File:

run this command directly after a clone

```
git status
On branch master
Your branch is up-to-date with 'origin/master'.  #告知所在分支
nothing to commit, working tree clean  #none of tracked files are modified
```

- 新增一个文件并查看状态

代码块

```
echo 'My Project' > README
#create a text and then write into File"README", if there is not such a file, a new file will be created
git status
On branch master
Your branch is up-to-date with 'origin/master'.
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README
nothing added to commit but untracked files present (use "git add" to track)
```

- Then we tracked this new file

代码块

```
git add README
git status
#feedback:
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)

    new file:   README
```

4.3 Staging Modified Files

Example: 修改名为CONTRIBUTING.md的文件

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
```

```
  (use "git reset HEAD <file>..." to unstage)

    new file:   README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md   #代表这个文件被修改了但没有执行ADD进入staging area
```

- 想要暂存此次更新，需要运行 `git add` 命令（multi-function: track new file/put tracked file in staging area）"精确地将内容添加到下一次提交中"

代码块

```
$ git add CONTRIBUTING.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README
    modified:   CONTRIBUTING.md
```

- 如果此时再次修改CONTRUBUTING.md文件未使用 `git add` 并查看status,会有一个"to be committed"的.md 和一个"not stage for commit"的.md（同时出现在暂存区、非暂存区)
- 此时如果提交是commit pervious version (last run `git add` version)

4.3.1 Short Status

- While `git status` output is pretty detailed, but it is quite wordy (too many words)
- `git status -s` or `git status --short` can be used to get a far more simplified output
- **Core:** there are two columns of status in the output (**XY Filename**, X is staging area status, Y is working tree status)

代码块

```
$ git status -s
 M README
MM Rakefile
A  lib/git.rb
M  lib/simplegit.rb
?? LICENSE.txt
#??: files aren't tracked
#A: files have been added to staging area
#M: Modified files
```

Question: 如果左边代表暂存区状态，在暂存区中的已修改和已暂存有什么区别?

4.3.2 Ignoring Files

- For those files which you don't want Git to auto add or even show you as being untracked (Some generally auto generated files such as log files/ files by build system) *#不想让Git跟踪，也不想每次* `git status` *都看到*
- **Solution:** create a file listing patterns to match them named `.gitignore`

代码块

```
$ cat .gitignore
*.[oa]   #忽略以.o和.a结尾的文件
*~      #忽略以 ~作为结尾的文件
```

- `.gitignore` 格式规范：

1. 所有 blank lines or lines starting with # will be ignored
2. Standard glob patterns work (支持通配符模式--可以写一些匹配规则)
3. Can start patterns with a forward slash"/" to avoid recursivity (防止"递归")
4. Can end patterns with a forward slash"/" to specify a directory(后加/，表示这是目录) `temp/`
5. Starting with an exclamation point "!" to negate a pattern(在已经ignore的情况下，忽略ignore 继续跟踪)

- 什么是glob patterns: **simplified regular experssion**s that shells use

代码块

```
*: matches zero or more characters
[abc]: matches any character inside brackets[] #a,b or c
[0-9]: matches all character between the range of a and b
?: matches a single character
**(asterisks): matches nested directories; #a/**/z would matches a/z, a/b/z, a/b/c/z...
```

4.3.3 Viewing Your Staged and Unstaged Changes

- `get diff` observe which lines experience changes in detail by listing file name (只能看尚未暂存的文件)
  - **Compare** what is in the working directory with what is in your staging area
  - Result tells you the **changes you made but not staged**
  - If all of changes are staged, `get diff` will give **no output**
- `git diff --staged` compares staged changed to then **last commit** --to see what you've staged will go into **next commit** (查看已暂存、将添加到下次提交里的内容)
- `git diff --cached` to see what you've staged so far

4.3.4 Commit changes

- Remember: files have been created or modified without running `git add` won't go into commit
- `git commit` : default commit message(注释行) contains latest output of `git status` and one empty line on top(type your commit message 输入提交声明)

> *: matches zero or more characters [ab…

- 退出编辑器时，Git会丢弃注释行，用输入的提交声明生成一次提交
- 可以使用 `git commit -m` 将提交信息与 `Git commit` 放在同一行,不用单独输入

4.3.5 Skip the staging area

- Staging Area: crafting commits exactly how you want them, but sometimes a bit more **complex** in workflow
- `git commit -a` auto staging the **tracked files** then commit--no longer need `git add`

4.3.6 Removing Files

- To remove files from Git, you have to remove it from the **staging area** and then **commit**
- 如果只是"手动删除文件"：status查询会显示 "Changes not staged for commit" (only change the working tree)
- `git rm "File name"` 下一次提交时该文件不在纳入版本管理 (additionally change the **staging area**)
- `git rm -f "File name"` 强制删除，用于删除之前修改过或已经放到暂存区的文件 *#有些内容没保存但是想直接删除，Git觉得不妥的情况下需要使用这个指令*
- `git rm --cached "File name"` 用于删除staging area的文件，但是保留当前working tree中的该文件
  - particularly useful if you forgot to add something to your `.gitignore` file and **accidentally staged it**, like a large log file or a bunch of `.a` compiled files (暂存了不想要的文件)
- `git rm` can pass files, directories and file-glob patterns (通过通配符删掉一批符合格式文件)

代码块

```
$ git rm log/\*.log  #removes all files whose name end with a ".log" under the directory "log/"
$ git rm \*~  #removes all files whose name end with a "~"
```

4.3.7 Moving Files

- Git **doesn't explicitly track** file movement (no metadata in Git tells you the movement)
- `git mv file_from file_to` 把file_from 改名或移动成 file_to,并把变化存入staging area
  - After that, `git status` will find:

```
$ git mv README.md README
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    README.md -> README
```

  - 将“改名” “delete staging” “adding staging” 三条命令合在一起

代码块

```
mv README.md README #Rename
git rm README.md   #Yield the old version is deleted
git add README   #Yield this new file should be tracked
```

4.4 Viewing the Commit History

4.4.1 View commit history

- `git log` 按时间先后顺序列出所有提交，最近的更新在**最上面**、
- 命令列出：
  - 每个提交的SHA-1校验和
  - 作者名字和邮箱地址
  - 提交时间
  - 提交说明
- `git log -p` / `git log --patch`
  - 显示提交时引入的差异

代码块

```
- s.version = "0.1.0" #"-"开头 代表被删除的旧内容
+ s.version = "0.1.1" #"+"开头 代表新增的新内容
```

  - 限制现实的日志条目数量 `git log --patch -2` 代表只显示最近的两次提交
- `git log --stat` 列出：
  - 所有被修改的文件
  - 多少文件被修改了
  - 哪些行被移除或者添加了

代码块

```
#本次提交修改了Rakefile; 一共改了2行; 新增1行, 删除1行
Rakefile | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
#本次提交修改了lib/simplegit.rb; 一共改了5行; 删除5行
lib/simplegit.rb | 5 -----
1 file changed, 5 deletions(-)
```

- `git log --pretty=xxx(子选项)` 使用不同于默认格式的方式展示提交历史
子选项：short, full, fuller, oneline....
- `git log --pretty=oneline` 把每一个提交**放在一行内**显示

代码块

```
# 提交哈希 提交说明
ca82a6d... changed the version number
085bb3b... removed unnecessary test
a11bef0... first commit
```

- `git log --pretty=format:""` 定制记录的**显示格式**(有利于后期提取分析)

代码块

git log 按时间先后顺序列出所有提交，最…

段竣瀚 4月14日 17:21

命令列出：

段竣瀚 4月14日 16:38

git log --pretty=format:"" 定制记录的显…

段竣瀚 4月14日 17:12

```
git log --pretty=format:"%h - %an, %ar : %s"
#%h: 短哈希值 %an: 作者名字 %ar: 相对时间，比如 “6 years ago” %s: 提交说明
ca82a6d - Scott Chacon, 6 years ago : changed the version number
085bb3b - Scott Chacon, 6 years ago : removed unnecessary test
```

- `git log --pretty=format:"" --graph` 添加ASCII字符串来形象展示分支与合并历史

  代码块

  ```
  "*" commit 节点
  "|" 表示历史线仍在继续
  "\""/" 通常与"|"合用， 一起表示分支分叉或会合 （开支线）
  ```

4.4.2 Limiting Log Output

1. `-<n>` 只看近n次的提交
2. `git log --since` 按**时间**筛选，某个时间之后的提交
   a. 直接接值：`git log --since=2.weeks` **#只看最近两周的提交**
   b. 写具体日期：`git log --since="2024-01-15"` **#看从2024.01.15到现在的提交**
   c. 写相对时间：`git log --since="2 years 1 day 3 minutes ago"` **#看从两年零一天三分钟前到现在的提交**
3. `git log --until` 按**时间**筛选，某个时间之前的提交
   a. `git log --until="2024-01-15"` **#看2024.01.15前的提交**
   b. 两者合用，框出时间范围：`git log --since="2024-01-01" --until="2024-01-31"` **#只看24年一月**
4. `git log --author="Junhan"` 按**作者**筛选
5. `git log --grep="fix"` 按提交的**关键词**筛选
   a. `git log --grep="fix" --grep="bug"` **多个词**一起筛选
   b. `git log --grep="fix" --grep="bug" --all-match` **多个词**一起筛选，且必须**同时满足**
6. `git log -S function_name` 根据字符串function_name出现的次数变化

git log --pretty=format:"" --graph 添加A…

段竣瀚 4月14日 17:21

git log -S function_name 根据字符串fun…

段竣瀚 4月15日 08:44

？？？

6. `-S`：按“某个字符串的出现次数变化”来找提交

这个很重要，图里也特别强调了。

例如：

```bash
git log -S function_name
```

意思是：

查找那些让字符串 `function_name` 的出现次数发生变化的提交

也就是找：

- 哪次把这个字符串加进来了
- 哪次把这个字符串删掉了

这特别适合找：

- 某个函数名第一次出现在哪次提交
- 某段配置是哪次被删掉的
- 某个变量是哪次引入的

你可以把它理解成：

按“字符串是否被增减”找提交

7. `git log -- path/to/file` 路径过滤
   a. 前面是选项，后面是路径；
   b. eg:最近两周谁改了路径README `git log --since=2.weeks -- README.md`

4.5 Undoing Things 撤销操作

4.5.1 Undoing

- `git commit --amend` commit again 重新提交
  - Use the staging area for commit
  - If not make changes since last commit, snapshot looks the same, only change the commit message

代码块

```
$ git commit -m 'Initial commit' #首次提交
$ git add forgotten_file  #修改文件
$ git commit --amend   #再次提交
```

4.5.2 Unstaging a staged File 取消暂存文件

- Background: you have changed two files and are willing to commit them separately as two independent modification, but input `git add *` to **stage both of them**
- `git reset HEAD <file>` Unstage a specific file

git reset HEAD <file> Unstage a specific ···

段竣瀚 4月15日 08:57

git reset --hard 直接改动工作区和历史，风险大

4.5.3 Unmodifying a Modified File

- Discard the modifications at the **working directory** and revert it back to the file version in the **staging area**
- `git checkout -- <file>` 撤销之前所做的修改
- `Git restore --staged <file>` 2.23.0 version Git use this instead of `git reset HEAD` ***#从暂存区移除，保留工作区修改***
- `git restore <file>` #撤销之前做的修改，同 `git checkout -- <file>`

4.6 Working with Remotes 远程仓库

- Defination of **Remote Repositories**: Versions of project that are hosted on the Internet or network somewhere. 一些对用户只读，一些可以读写

4.6.1 Showing your Remotes 查看

- `git remote` lists the shortnames of each remote handle you've specified ----克隆仓库后至少会看见Git给server的默认名称 **origin**
- `git remote -v` lists the URLs(网址) that Git has stored for the shortame when reading and writing to the remote
  - If a repository has multiple remotes,command lists them all
  - Fetch: 代表 拉取/读取 远程仓库内容时 使用的远程地址
  - Push: 代表 写入/推送 远程仓库内容时 使用的远程地址

代码块

```
$ git remote -v
origin        https://github.com/schacon/ticgit (fetch)
origin        https://github.com/schacon/ticgit (push)
```

4.6.2 Adding Remote Repositories 添加库

- `git remote add <shortname> <url>` 添加新的远程Git仓库并指定一个简写

代码块

```
$ git remote
origin
$ git remote add pb https://github.com/paulboone/ticgit
$ git remote -v
origin        https://github.com/schacon/ticgit (fetch)
origin        https://github.com/schacon/ticgit (push)
pb        https://github.com/paulboone/ticgit (fetch)
pb        https://github.com/paulboone/ticgit (push)
```

- `git fetch <shortname>` pull down the branches infor and pull down the **latest commit** among **those branches** (获取其他人仓库中有但你没有的信息)

4.6.3 Fetching and Pulling from your Remotes

- When you run `git fetch <remote_shortname>` ,the command will:
  - **pulls down** all the data from that remote project that you **don’t have yet**
  - have **references** to all the branches from that remote, which you can merge in or inspect

- You have to **merge it manually** into your work when you’re ready

- `git pull` automatically **fetch and merge** that remote branch into your current branch

4.6.4 Pushing to Your Remotes

- `git push <remote> <branch>` If you want to share, you have to push it upstream

Example

```
# If you want to push your master branch to your origin server
#(again, cloning generally sets up both of those names for you automatically),
#then you can run this to push any commits you’ve done back up to the server:
$ git push origin master
```

- **tips**: 如果别人已经先把新提交推到远程了，你的本地分支就落后了；这时不能直接push，必须先把远程更新拉下来并合并，再重新推送

4.6.5 Inspecting a Remote

- `git remote show <remote>` see information about a **particular remote**
  - Shows which branch is auto pushed when run `git push`
  - Which remote branches on server you don't yet have
  - Which branches you have that have been removed from server
  - Multiple local branches are able to merge auto with their remote tracking branch(某个分支上次的已知状态) when run `git pull`

4.6.6 Renaming and Removing Remotes

- `git remote rename` 修改远程仓库名

代码块

```
$ git remote rename pb paul
$ git remote
origin
paul
```

- `git remote remove` `git remote rm` 移除远程仓库
  - 同时删除相关远程追踪以及配置信息

代码块

```
$ git remote remove paul
$ git remote
origin
```

4.7 Tagging 打标签

4.7.1 List your Tag

- type `git tag` (with optional `-l` or `-list`) to list the existing tags in Git

git remote show <remote> see informa…

段竣瀚 4月15日 10:58

Use git tag -l "tag_name" to search a s…

段竣瀚 4月15日 11:22

- Use `git tag -l "tag_name"` to search a specific tags

4.7.2 Creating Tags

- Git supports **two types of tags:**
  - Lightweight tag: A **pointer** to a specific commit, doesn't change
  - Annotated tag: Stored as full objects in the Git database (They’re checksummed; contain the tagger name, email, and date; have a tagging message; and can be signed and verified with GNU Privacy Guard (GPG))

4.7.3 Annatated Tags "-a" 附加标签

- `git tag -a tag_name -m "specific tagging message"`
- If you **don’t specify a message** for an annotated tag, Git launches your editor so you can type it in
- `git show tag_name` shows the **tagger information**: the date the commit was tagged, the annotation message before showing the commit information

4.7.4 Lightweight Tags

- Commit checksum stored in a file **without other information is kept**
- **Don't** provide any of the "-a, -s, -m", but **just a tag name**

代码块

```
$ git tag v1.4-lw
$ git tag
v0.1
v1.3
v1.4
v1.4-lw
v1.5
```

- `git show tag_name` just show the commit **without** extra tag information

代码块

```
$ git show v1.4-lw
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Mar 17 21:52:11 2008 -0700

    Change version number
```

4.7.5 Tagging Later 后期打标签

- `git tag -a v1.2 9fceb02` "-a": 创建附注标签 "v1.2": 标签名 "9fceb02": 标签打到哪次提交上，这一般是缩短的整个标签的前几位

代码块

```
$ git log --pretty=oneline
15027957951b64cf874c3557a0f3547bd83b3ff6 Merge branch 'experiment'
a6b4c97498bd301d84096da251c98a07c7723e65 Create write support
0d52aaab4479697da7686c15f77a3d64d9165190 One more thing
6d52a271eda8725415634dd79daabbc4d9b6008e Merge branch 'experiment'
0b7434d86859cc7b8c3d5e1dddfed66ff742fcbc Add commit function
4682c3261057305bdd616e23b64b0857d832627b Add todo file
```

```
166ae0c4d3f420721acbb115cc33848dfcc2121a Create write support
9fceb02d0ae598e95dc970b74767f19372d61af8 Update rakefile
964f16d36dfccde844893cac5b347e7b3d44abbc Commit the todo
8a5cbc430f1a9c3d00faaeffd07798508422908a Update readme
```

代码块

```
$ git tag -a v1.2 9fceb02 #找到9fce在上面对应的词，打到1.2上
#####
$ git tag
v0.1
v1.2
v1.3
v1.4
v1.4-lw
v1.5

$ git show v1.2
tag v1.2
Tagger: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Feb 9 15:32:16 2009 -0800

version 1.2
commit 9fceb02d0ae598e95dc970b74767f19372d61af8
Author: Magnus Chacon <mchacon@gee-mail.com>
Date:   Sun Apr 27 20:43:35 2008 -0700

    Update rakefile
```

4.7.6 Sharing Tags

- `git push origin <tagname>` transfer tags to remote servers

Example

```
$ git push origin v1.5
Counting objects: 14, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (12/12), done.
Writing objects: 100% (14/14), 2.05 KiB | 0 bytes/s, done.
Total 14 (delta 3), reused 0 (delta 0)
To git@github.com:schacon/simplegit.git
 * [new tag]         v1.5 -> v1.5
```

- `git push origin --tags` transfer a lot of tags

Example

```
$ git push origin --tags
Counting objects: 1, done.
Writing objects: 100% (1/1), 160 bytes | 0 bytes/s, done.
Total 1 (delta 0), reused 0 (delta 0)
To git@github.com:schacon/simplegit.git
 * [new tag]         v1.4 -> v1.4
 * [new tag]         v1.4-lw -> v1.4-lw
```

4.7.7 Deleting Tags

- `git tag -d <tagname>` delete a tag on your local repository
- `git push origin --delete <tagname>` delete a tag from a **remote server**
- `git push <remote> :refs/tags/<tagname>` delete a tag from a **remote server**

4.7.8 Checking out Tags

- `git checkout <tagname>`
- detached HEAD state:
  - if you make changes and then create a commit, the tag will stay the same, but your new commit won’t belong to any branch and will be unreachable, except by the exact commit hash
  - if you need to make changes — say you’re fixing a bug on an older version, for instance — you will generally want to create a branch 创建新的分支

4.8 Git Aliases Git 别名

- Core: 自定义缩写
- Set up an aliases for each command using `git config`

代码块

```
$ git config --global alias.co checkout
$ git config --global alias.br branch
$ git config --global alias.ci commit
$ git config --global alias.st status
```

- Instead of type `git commit`, you just need to type `git ci`
- `git config --global alias.unstage 'reset HEAD --'`

定义一个 "`git unstage`" 等价于 "`git reset HEAD --`"

Practice Issues

1. 既然桌面中加入git管理的文件不可以管理桌面上其他独立文件，那为什么 我在其中一个文件 git_test中的命名与邮箱 会在另外一个被init的文件中查看到
   a. 定义时加了 "--global

在Github中建立一个远端仓库

点击右上角的"**+**",选择 new repository

创建/切换目标branch

`git branch <branch_name>`

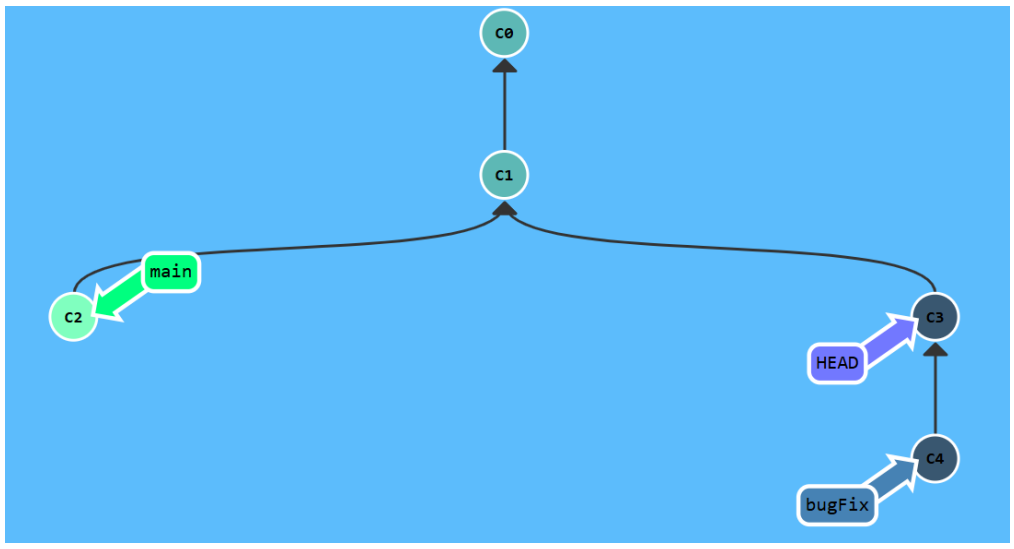
`git checkout <branch_name>`

二合一：`git checkout -b <branch_name>`

`git rebase <branch_name>` 把目前选中的branch接在另一条branch上

`<file_name>^` 寻找目前节点的parent--推理上一个节点：

`Git checkout bugfix^` 推理bugfix上一节点的**"HEAD"**



`~<num>` 用来表示后退几步

如果想像移动HEAD一样移动branch: `git branch -f main C6` 强行创造一个C6并转移main

`git branch -f bugFix HEAD~1` 看起来像是把bugFix想对HEAD上移1位

`git reset` 上移一格(C2-->C1),C2变为Unstaged

`git revert` 撤销更改，但是是通过引入新更改(C2')的方法,可以推送至远程分支与他人分享

`git cherry-pick C3 C4 C7` 用当前目标根据输入的file_name顺序展示想要的提交记录

`git rebase -i HEAD~4` 交互式rebase, 选择需要*include*的file_name并给他们排序，在当前选中的file的前4个作为分叉，接一条新的提交记录